

Apache Airflow

Training / Learning Documentation

Prepared by: **SHINY ZERUBBA RAJIVE S**

Date: 07-MAY-2026

Apache Airflow

Apache Airflow is an open-source workflow orchestration platform used to schedule, automate, and monitor data pipelines and workflows. It was originally created by Airbnb and later became a project of the Apache Software Foundation.

Airflow is widely used in:

- Data Engineering
- ETL Pipelines
- Machine Learning Workflows
- Cloud Automation
- Analytics Pipelines

Airflow allows users to define workflows programmatically using Python.

What is Workflow Orchestration?

Workflow orchestration means managing and coordinating multiple tasks in a defined sequence.

Example:

Extract Data → Transform Data → Load Data → Generate Report

Instead of manually running each step, Airflow automates the entire process.

Features of Apache Airflow

- Open-source platform
- Python-based workflow creation
- Easy scheduling and monitoring
- Scalable architecture
- Web-based User Interface
- Supports task dependencies
- Integration with many tools and cloud platforms
- Retry and failure handling mechanisms

Advantages of Apache Airflow

1. Open Source

Airflow is free to use and supported by a large community.

2. Python-Based

Workflows are written using Python, making them flexible and easy to customize.

Example:

```
task1 >> task2 >> task3
```

3. Easy Scheduling

Airflow supports:

- Daily jobs
- Hourly jobs
- Weekly jobs
- Custom schedules using Cron expressions

4. Scalability

Airflow can handle:

- Small workflows
- Large enterprise-level pipelines

It supports distributed execution using multiple workers.

5. Monitoring and Logging

Airflow provides:

- Real-time monitoring
- Task execution history
- Detailed logs
- Failure alerts

6. Extensible Architecture

Airflow integrates with:

- Apache Spark
- [Databricks](#)
- [Amazon Web Services](#)

- [Google Cloud Platform](#)
- [Microsoft Azure](#)
- Databases
- APIs
- Docker
- Kubernetes

7. Fault Tolerance

Airflow can automatically:

- Retry failed tasks
- Resume workflows
- Send notifications for failures

Architecture of Apache Airflow

Airflow architecture consists of multiple components working together.

Main Components

1. Web Server
2. Scheduler
3. Metadata Database
4. Executor
5. Worker
6. DAG Directory

1. Web Server

The Web Server provides the Airflow User Interface (UI).

Functions:

- View DAGs
- Trigger workflows
- Monitor task status
- Check logs
- Manage workflow execution

The UI acts like the control tower of the workflow airport ✈️

2. Scheduler

The Scheduler is the core component of Airflow.

Responsibilities:

- Monitors DAGs
- Determines task execution time
- Checks dependencies
- Sends tasks to the executor

Without the Scheduler, workflows will not run.

3. Metadata Database

The Metadata Database stores:

- DAG information
- Task states
- Logs metadata
- User information
- Scheduling details

Common databases used:

- SQLite
- PostgreSQL
- MySQL

4. Executor

The Executor determines how tasks are executed.

Responsibilities:

- Executes tasks
- Distributes tasks to workers
- Manages parallel processing

Types of Executors:

- SequentialExecutor
- LocalExecutor
- CeleryExecutor
- KubernetesExecutor

5. Workers

Workers perform the actual execution of tasks.

Example:

- Running Python scripts
- Executing SQL queries
- Calling APIs
- Processing files

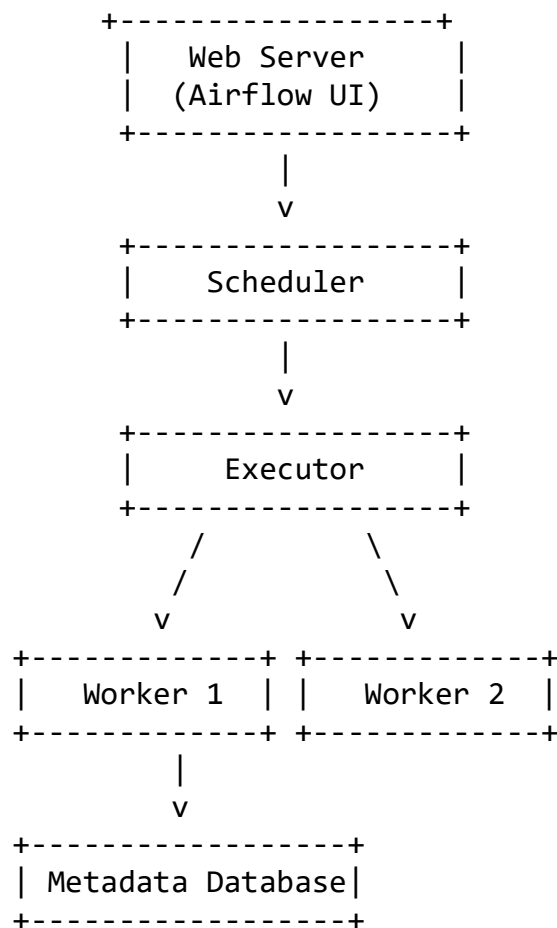
In distributed environments, multiple workers execute tasks simultaneously.

6. DAG Directory

The DAG Directory contains Python files that define workflows.

Airflow continuously scans this directory to identify new DAGs.

Airflow Architecture Diagram



DAG in Apache Airflow

What is a DAG?

DAG stands for:

Directed Acyclic Graph

A DAG represents a workflow in Airflow.

It defines:

- Tasks
- Dependencies
- Execution order

Meaning of DAG

Directed

Tasks move in a specific direction.

Example:

Task A → Task B → Task C

Acyclic

The workflow cannot loop back.

Invalid Example:

A → B → C → A

This creates an infinite cycle and is not allowed.

Graph

The workflow is represented visually as connected tasks.

Example of a DAG

Extract Data



Transform Data



Load Data



Generate Report

DAG Components

A DAG contains:

Component	Description
DAG ID	Unique name of workflow
Tasks	Units of work
Operators	Define task type

Dependencies	Execution order
Schedule	When DAG runs

Executor in Apache Airflow

What is an Executor?

An Executor is responsible for running tasks.

It decides:

- Where tasks run
- How tasks run
- How many tasks run simultaneously

The Executor acts like a traffic manager for workflow execution 🚦

Types of Executors

1. SequentialExecutor

- Default executor for beginners
- Executes one task at a time
- Mainly used for testing and learning

Advantages:

- Simple setup

Disadvantages:

- No parallel execution

2. LocalExecutor

- Executes multiple tasks in parallel
- Runs on a single machine

Advantages:

- Faster than SequentialExecutor

Disadvantages:

- Limited scalability

3. CeleryExecutor

- Distributed executor
- Uses multiple worker machines

Advantages:

- Highly scalable
- Parallel execution

Disadvantages:

- More complex setup

4. KubernetesExecutor

- Executes tasks inside Kubernetes pods

Advantages:

- Dynamic scaling
- Cloud-native architecture

Disadvantages:

- Requires Kubernetes knowledge

Comparison of Executors

Executor	Parallelism	Scalability	Complexity
SequentialExecutor	No	Low	Easy
LocalExecutor	Yes	Medium	Moderate
CeleryExecutor	Yes	High	High
KubernetesExecutor	Yes	Very High	Advanced

Conclusion

Apache Airflow is a powerful workflow orchestration platform used to automate and manage data pipelines efficiently. Its scalable architecture, Python-based workflows, scheduling capabilities, and monitoring features make it one of the most popular tools in modern data engineering and cloud-based workflows.

Key concepts such as DAGs and Executors form the foundation of Airflow and help manage workflow execution systematically and efficiently.